

Белорусский Государственный Университет
Факультет прикладной математики и информатики

Компьютерная графика

Кулинкович Виктория Александровна



Базовые растровые алгоритмы

- Векторная и растровая графика.
- Постановка задачи *растеризации*.
- Изображение отрезка с целочисленными координатами концов.
- Цифровой дифференциальный анализатор.
- Алгоритм Брезенхема.
- Алгоритм Кастла-Питвея.
- Изображение отрезка с нецелочисленными координатами концов.
- Изображение окружностей. Алгоритм Брезенхема.

Векторная графика и растровая графика

Векторная графика — способ представления объектов и изображений, основанный на использовании геометрических примитивов (таких как точки, линии, сплайны, многоугольники), заданных своим математическим описанием.

Растровая графика — способ представления объектов, основанный на использовании сетки пикселей.



Растровая графика

• Эффективно представляет реальные образы.

• Обеспечивает высокую точность передачи градаций цветов и полутонов.

• Хорошо подходит для устройств вывода (принтеры, мониторы)

• Занимает большое количество памяти.

• Плохо масштабируется.



Векторная графика

• Занимает меньшее количество памяти.

• Гибкая масштабируемость.

• Возможность легкого редактирования всех частей векторного изображения;

• Простой экспорт векторного рисунка в растровый.

• Отсутствие реалистичности.

• Невозможность использования эффектов, которые можно применять в растровой графике.

• Практически полная невозможность экспорта растрового рисунка в векторный.



Введение в растеризацию кривых

Возникающая при выводе задача отображения геометрических объектов, заданных их математическим описанием, на растре, называется **растеризацией**.

Растеризация – процесс определения пикселей, наилучшим образом аппроксимирующих заданный отрезок/кривую.

Критерии алгоритмов растеризации:

- качество
- скорость
- простота реализации



Введение в растеризацию кривых

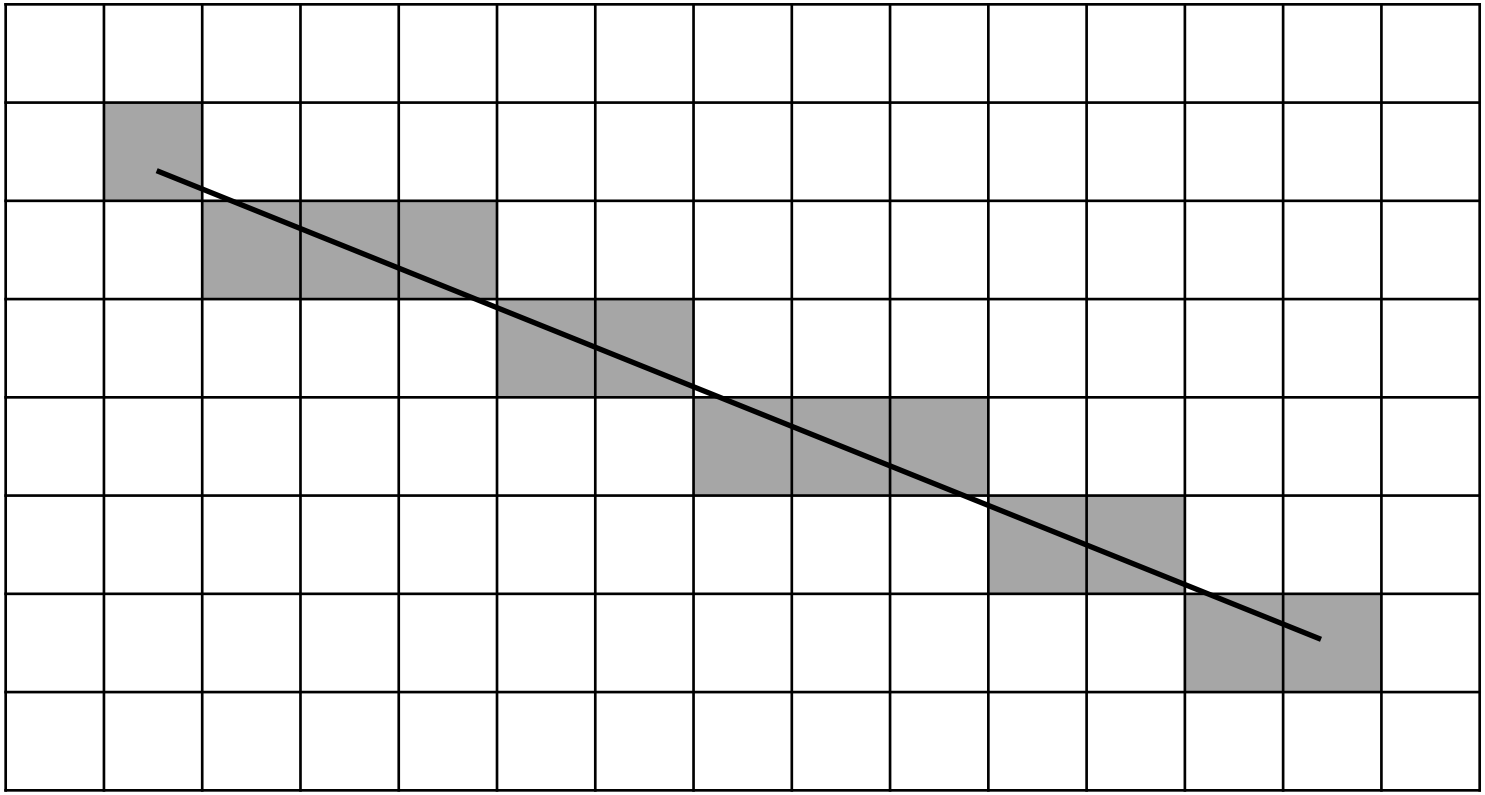
CPU vs GPU

Одна задача растеризации — два подхода

CPU
Последовательный обход кривой с шагом в один пиксель.
На каждом шаге — одна целочисленная операция.
Основа — алгоритм Брезенхэма (1962, IBM).

GPU
Геометрия декомпозируется на треугольники.
Параллельная проверка каждого пикселя:
принадлежит ли точка (x, y) треугольнику?
Число потоков — порядка тысяч, выполняются одновременно.

Растеризация





Понятие линии

В качестве **линии** на растровой сетке выступает набор пикселей

$$P_1, P_2, \ldots, P_n$$

где любые два пикселя P_i и P_{i+1} являются соседними.

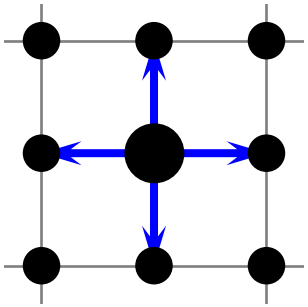


Понятие связности

Понятие линии базируется на понятии **связности** – возможности соединения двух пикселей растровой линией. При этом возникает вопрос, когда пиксели (x1,y1) и (x2,y2) можно считать соседними. Вводится два понятия связности.

Понятие связности

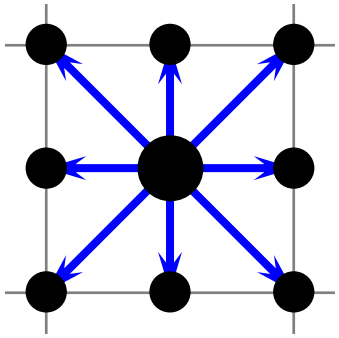
4-связность, когда пиксели считаются соседними, если либо их x-координаты, либо y-координаты отличаются на единицу



$$|x_1 - x_2| + |y_1 - y_2| \leq 1$$

Понятие связности

8-связность, когда пиксели считаются соседними, если их x- и y-координаты отличаются не более чем на единицу, то есть:



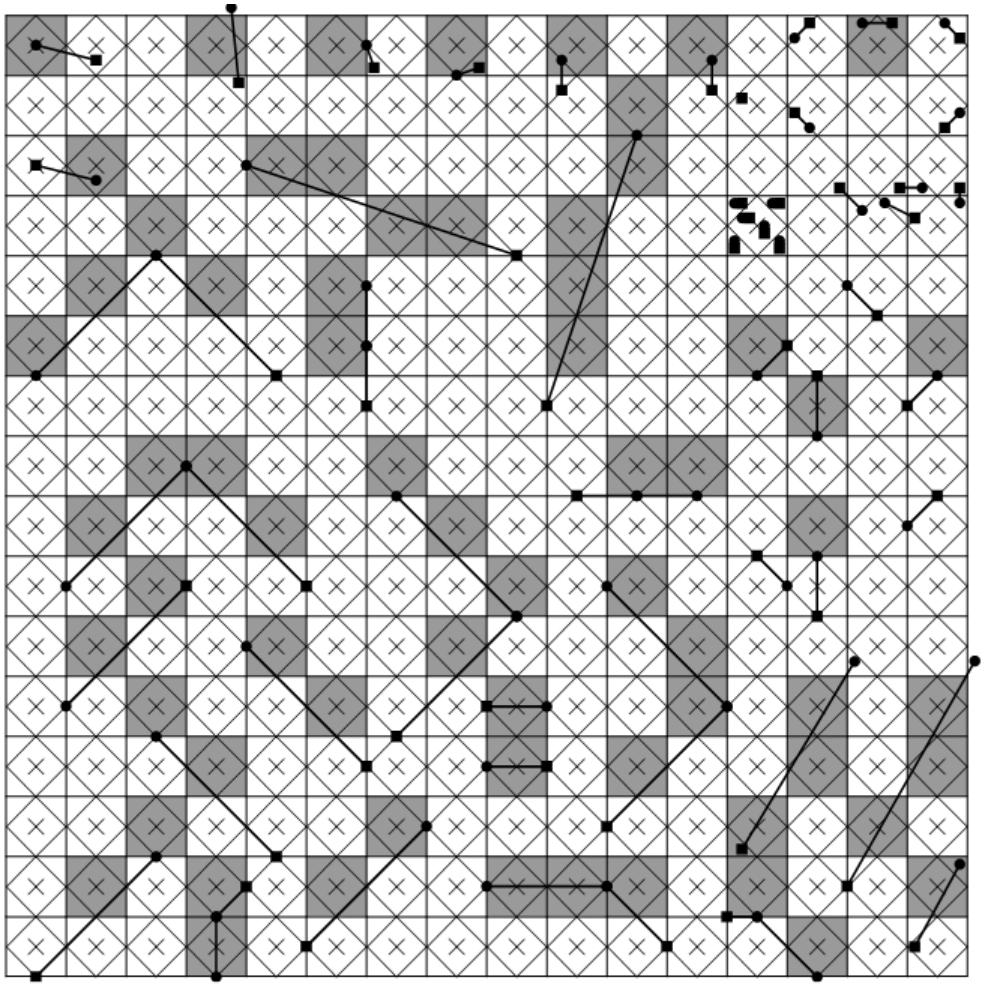
$$|x_1 - x_2| \le 1, \quad |y_1 - y_2| \le 1$$



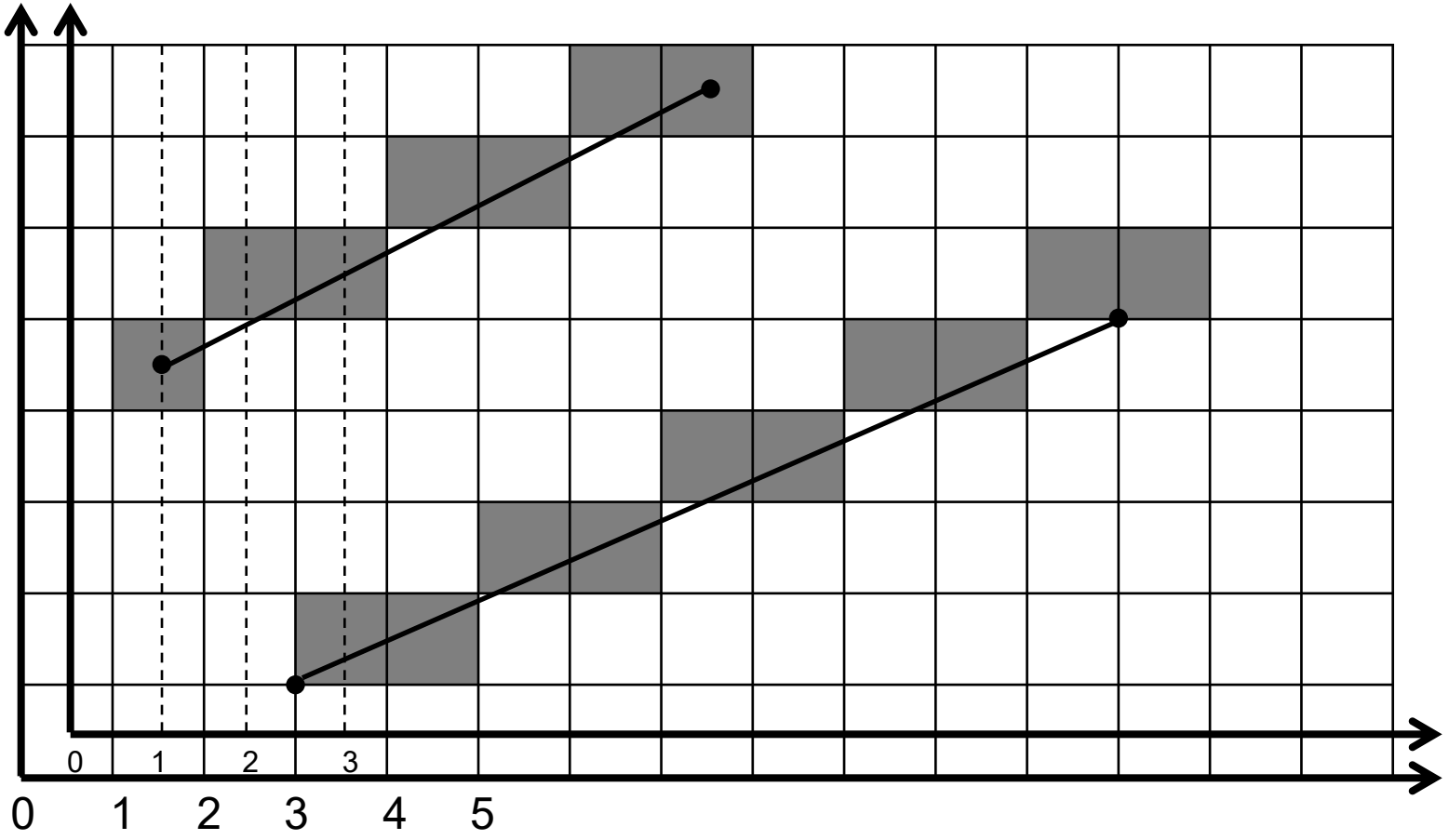
Понятие связности

Понятие ***4-связности*** является более сильным, чем ***8-связность***: любые два 4-связных пиксела являются и 8-связными, но не наоборот.

Изображение отрезка с нецело-численными координатами концов



Изображение отрезка с целочисленными координатами концов





Изображение отрезка с целочисленными координатами концов

- Пошаговый алгоритм
- Алгоритм цифрового дифференциального анализатора
- Алгоритм Брезенхема
- Алгоритм Кастла-Питвея

Пошаговый алгоритм

- Наиболее очевидный.
- Заключается в постепенном увеличении координаты ***x***, вычислении координаты ***y=kx+b*** и последующем округлении полученного результата до целого числа.

Пошаговый алгоритм

Недостатки:

- Работа с вещественными числами.
- Вычисление произведения kx и последующее округление

  → затрачивается много времени.

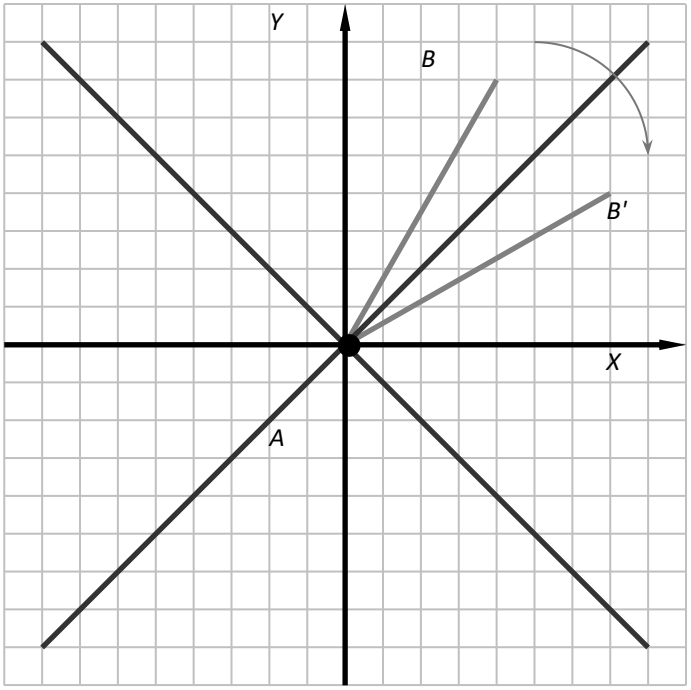


Алгоритм Брезенхема

- *Bresenham's line algorithm*.
- Строит 8-связную линию.
- Разработан Джеком Е. Брезенхемом в компании IBM в 1962 году.
- Был первоначально разработан для цифровых графопостроителей, однако он в равной степени подходит и для использования растровыми устройствами с ЭЛТ.

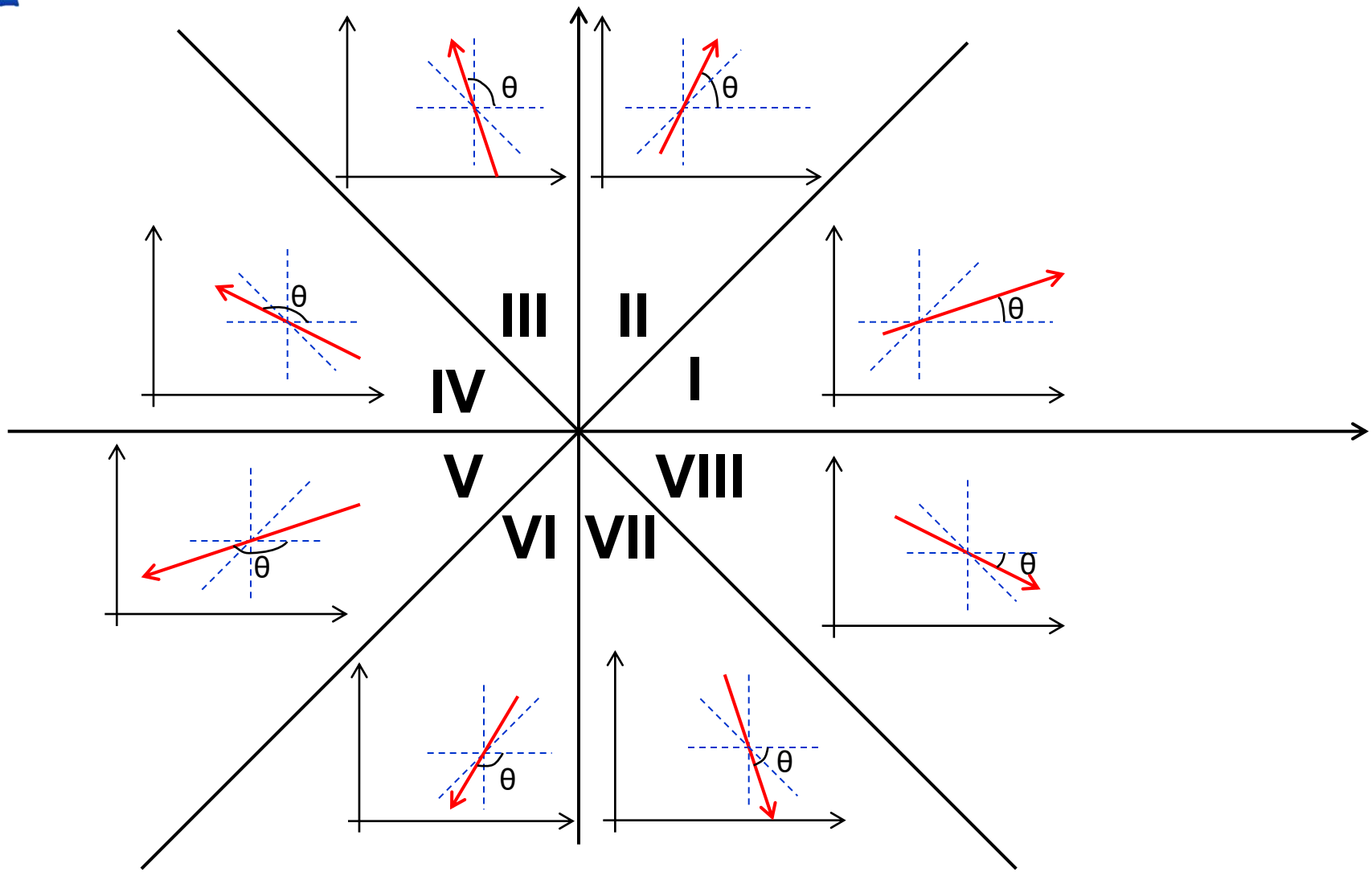
Алгоритм Брезенхема

Рассмотрим алгоритм Брезенхема на примере проведения отрезка, лежащего в первом октанте (***канонический случай***).



Отрезок может лежать в любом из 8 октантов, но всегда существуют симметрии относительно осей, разделяющих эти октанты, позволяющие свести задачу к случаю отрезка, лежащего в первом октанте.

Алгоритм Брезенхема



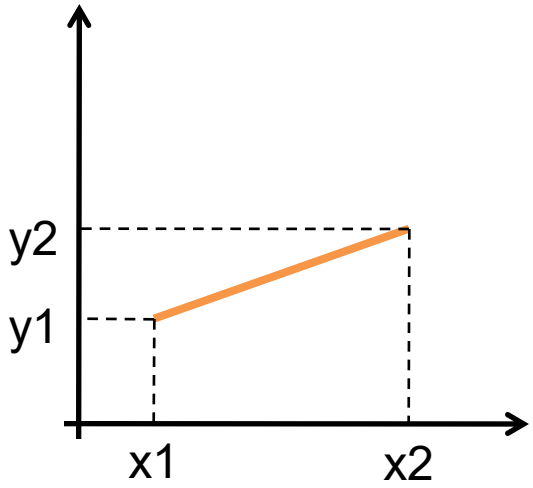
Алгоритм Брезенхема

Для отрезка, лежащего в первом октанте, справедливо следующее:

$$(x_1, y_1), (x_2, y_2)$$

$$x_2 > x_1, \quad y_2 > y_1$$

$$Dx = x_2 - x_1 > Dy = y_2 - y_1$$



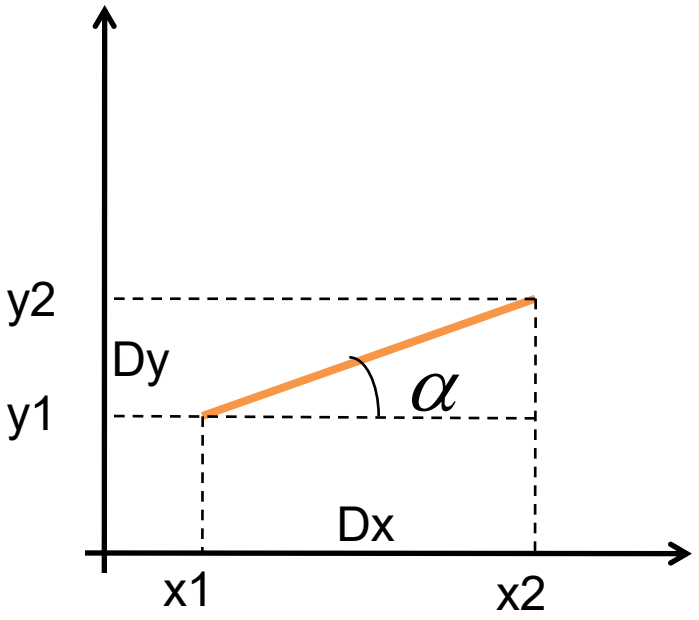
Алгоритм Брезенхема

$$y = kx + b$$

$$k = tg\alpha = \frac{Dy}{Dx}$$

$$0 \le \alpha \le \frac{\pi}{4}$$

$$0 \le k \le 1$$



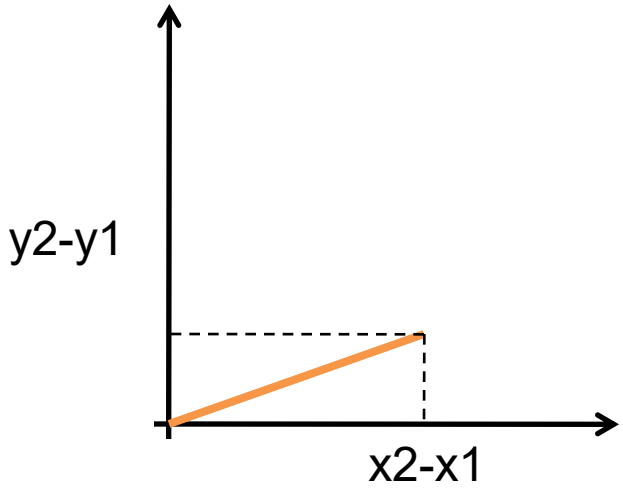
Алгоритм Брезенхема

Для отрезка, лежащего в первом октанте, справедливо следующее:

$$(x_1, y_1), (x_2, y_2)$$

$$\Rightarrow$$

$$(0,0), (x_2 - x_1, y_2 - y_1)$$



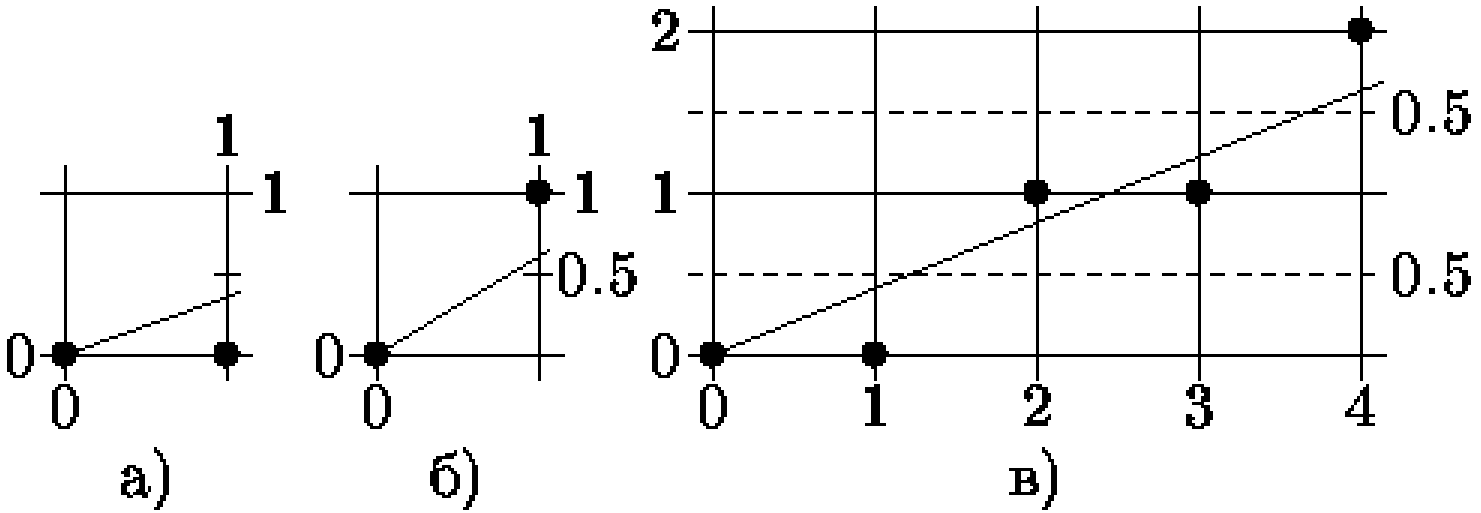


Алгоритм Брезенхема

- В процессе работы алгоритма координата x увеличивается на каждом шаге на 1, а координата y зависит от расстояния между действительным положением отрезка и ближайшими координатами сетки. Такое расстояние называется ошибкой.

Алгоритм Брезенхема

Если угловой коэффициент прямой < 1/2, то естественно точку, следующую за точкой (0,0), поставить в позицию (1,0), а если угловой коэффициент ⩾ 1/2, то - в позицию (1,1).



Алгоритм Брезенхема

Первоначально значение ошибки считается равным -½.

Текущие координаты (x,y)=(0,0)

На следующем шаге к ошибке прибавляется угловой коэффициент и анализируется её значение.

Если ошибка меньше *0*, то заполняется ячейка *(x+1, y)*, если больше 0, то заполняется ячейка *(x+1, y+1)*

Алгоритм Брезенхема

Ошибка для первого шага:

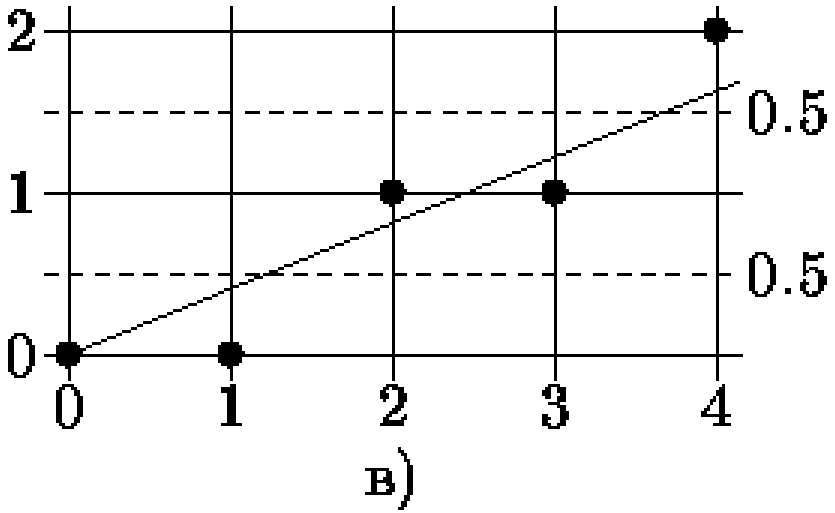
$$E1 = Dy/Dx - 1/2 < 0$$

→ Для занесения пикселя выбирается точка (1,0)

Ошибка для второго шага вычисляется добавлением приращения Y-координаты для следующей X-позиции

$$E2 = E1 + Dy/Dx > 0$$

→ Для занесения пикселя выбирается точка (2,1)



Алгоритм Брезенхема

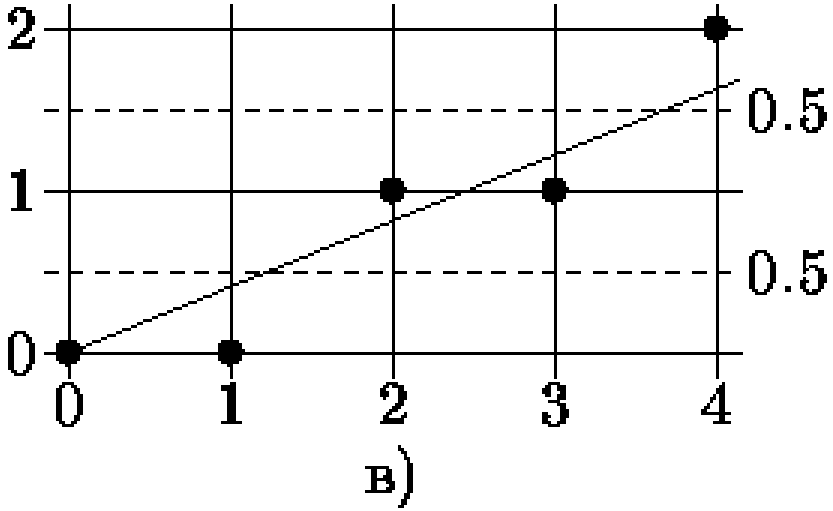
Ошибка для третьего шага:

$$E3 = E2 + Dy/Dx$$

Так как отклонение считается от Y-координаты, которая теперь увеличилась на 1, то из накопленного отклонения для вычисления последующих отклонений надо вычесть 1:

$$E3 = E3-1<0$$

→ Для занесения пикселя выбирается точка (3,1)



Алгоритм Брезенхема

```
X= x1;
Y= y1;
```
Приращение
```
Dx= x2 - x1;
Dy= y2 - y1;
```
Инициализация ошибки с поправкой на половину пикселя
```
E= Dy/Dx-1/2;
```
Первая точка вектора
```
PutPixel(X, Y);
```

```
while (X<=X2) {
        if (E⩾0) {
                X= X + 1;
                Y= Y + 1;
                E= E+Dy/Dx-1;
        } else {
                X= X + 1;
                E= E + Dy/Dx;
        }
```
Очередная точка вектора
```
        PutPixel(X, Y);
        }
```



Алгоритм Брезенхема

- Работа с вещественными числами
- Есть операция деления

- Разработана целочисленная модификация алгоритма

Целочисленный алгоритм Брезенхема

Быстродействие алгоритма можно увеличить если использовать только целочисленную арифметику и исключить деление.
Так как важен только знак ошибки, то введем преобразование:

$$E' = E*2*Dx$$

Целочисленный алгоритм Брезенхема

```
X= x1;
Y= y1;
Dx= x2 - x1;
Dy= y2 - y1;
E’= 2*Dy - Dx;
```

Первая точка вектора

```
PutPixel(X, Y);
```

```
while (X<=X2) {
        if (E’ ⩾0) {
                X= X + 1;
                Y= Y + 1;
                E’= E’ + 2*(Dy - Dx);
        } else {
                X= X + 1;
                E’= E’ + 2*Dy;
```

Очередная точка вектора

```
        PutPixel(X, Y);
        }
```

Целочисленный алгоритм Брезенхема

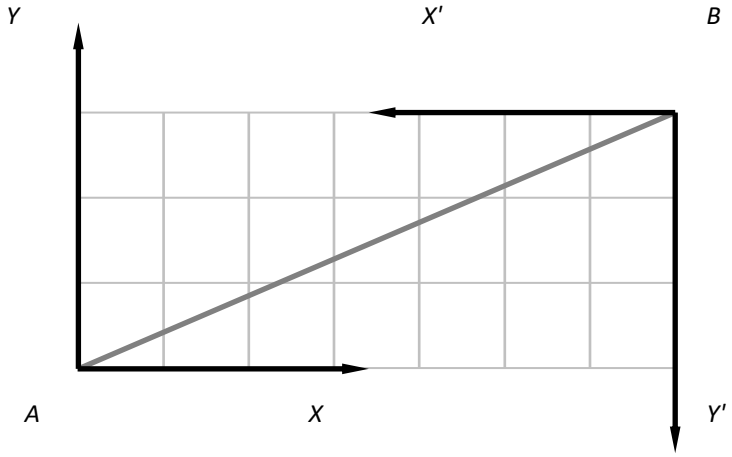
Таким образом получается алгоритм, в котором используются только целые числа, сложение, вычитание и сдвиг.

Этот алгоритм пригоден для случая 1го октанта: ***0<Dy<Dx***. Для других случаев алгоритм строится аналогичным образом.

```
void BresenhamLine(int x0, int y0, int x1, int y1)
{
    var steep = Math.Abs(y1 - y0) > Math.Abs(x1 - x0); // Проверяем рост отрезка по оси икс и по оси игрек
    // Отражаем линию по диагонали, если угол наклона слишком большой
    if (steep)
    {
        Swap(ref x0, ref y0); // Перетасовка координат вынесена в отдельную функцию для красоты
        Swap(ref x1, ref y1);
    }
    // Если линия растёт не слева направо, то меняем начало и конец отрезка местами
    if (x0 > x1)
    {
        Swap(ref x0, ref x1);
        Swap(ref y0, ref y1);
    }
    int dx = x1 - x0;
    int dy = Math.Abs(y1 - y0);
    int error = dx / 2; // Здесь используется оптимизация с умножением на dx, чтобы избавиться от лишних дробей
    int ystep = (y0 < y1) ? 1 : -1; // Выбираем направление роста координаты y
    int y = y0;
    for (int x = x0; x <= x1; x++)
    {
        DrawPoint(steep ? y : x, steep ? x : y); // Не забываем вернуть координаты на место
        error -= dy;
        if (error < 0)
        {
            y += ystep;
            error += dx;
        }
    }
}
```

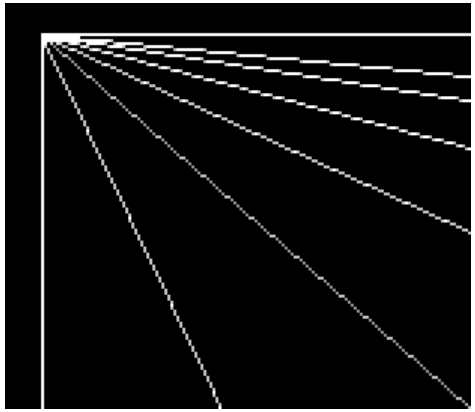
Оптимизация алгоритма Брезенхема

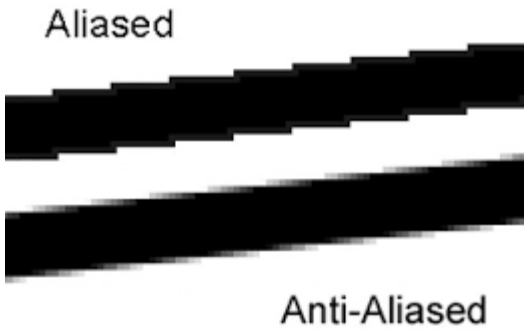
Оптимизация: отрезок симметричен относительно прямой, проходящей перпендикулярно ему через его середину => можно начинать рисовать сразу с двух концов, что сократит время общей работы алгоритма вдвое.



Рисование сглаженных линий

Сглаживание (anti-aliasing) — технология, используемая для устранения эффекта «зубчатости», возникающего на краях одновременно выводимого на экран множества отдельных друг от друга плоских или объёмных изображений.



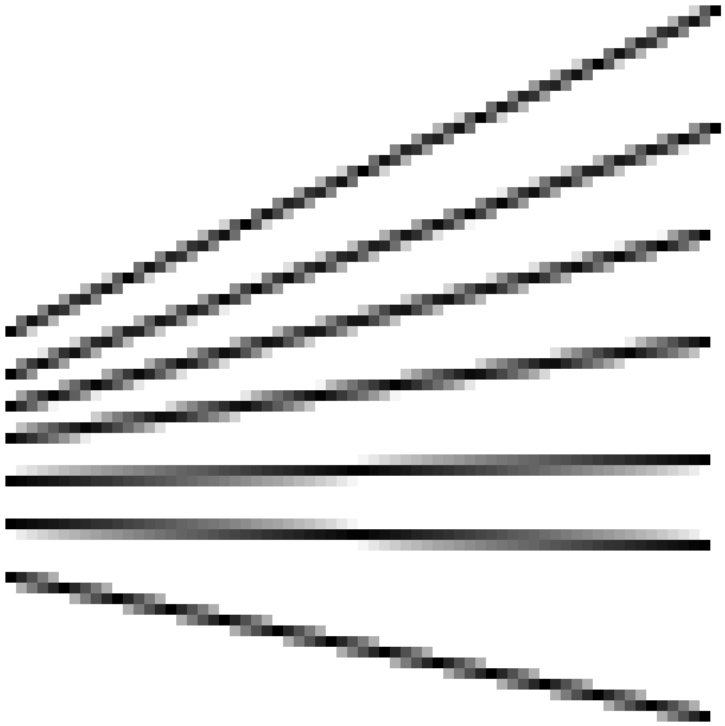


Как реализуется: пиксели, соседние с граничным пикселем изображения, принимают промежуточное значение между цветом изображения и цветом фона, создавая градиент и размывая границу.

Рисование сглаженных линий

Два подхода к сглаживанию:

- Растеризация уже сглаженных простейших объектов – линий и кривых (алгоритм Ву)

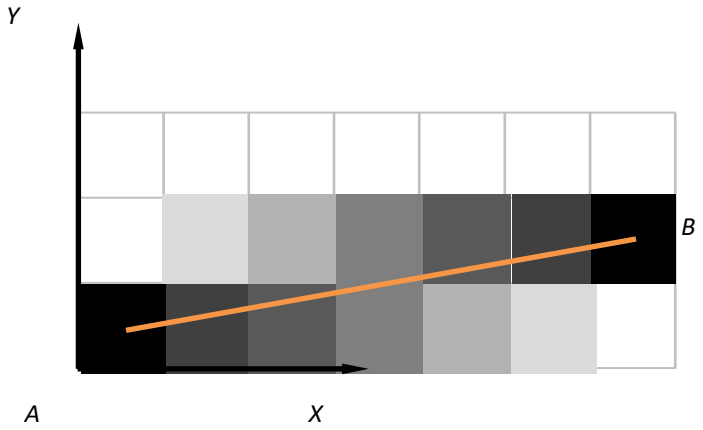


- Сглаживание уже отрисованного изображения (фильтрация)



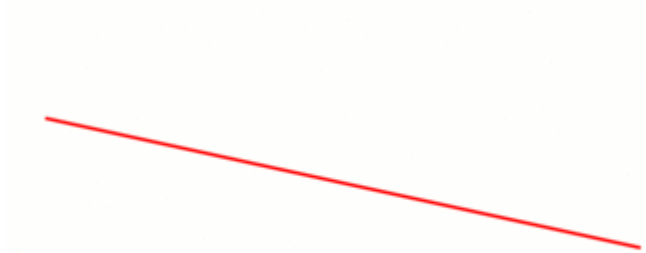
Алгоритм Ву для рисования сглаженных линий (Xiaolin Wu, 1991)

На каждом шаге ведётся расчёт для двух ближайших к прямой пикселей, и они закрашиваются с разной интенсивностью, в зависимости от удаленности. Точное пересечение середины пикселя даёт 100% интенсивности, если пиксель находится на расстоянии в 0.9 пикселя, то интенсивность будет 10%. Иными словами, сто процентов интенсивности делится между пикселями, которые ограничивают векторную линию с двух сторон.



Алгоритм Ву для рисования сглаженных линий

На каждом шаге ведётся расчёт для двух ближайших к прямой пикселей, и они закрашиваются с разной интенсивностью, в зависимости от удаленности. Точное пересечение середины пикселя даёт 100% интенсивности, если пиксель находится на расстоянии в 0.9 пикселя, то интенсивность будет 10%. Иными словами, сто процентов интенсивности делится между пикселями, которые ограничивают векторную линию с двух сторон.





Растеризация треугольников

Треугольник — базовый примитив 3D-графики.
Любая поверхность разбивается на треугольники.

Растеризация треугольников — это процесс преобразования треугольников, описывающих геометрию сцены, в набор пикселей на экране.



Цифровой дифференциальный анализатор (ЦДА)

- DDA - Digital *Differential Analyzer*.
- Строит 8-связную линию.
- Основан на определении дифференциала.
- Формирует дискретную аппроксимацию непрерывного решения дифференциального уравнения отрезка
- В настоящее время практически не применяется.

Цифровой дифференциальный анализатор (ЦДА)

$(x_1, y_1), (x_2, y_2)$ - конечные точки отрезка

$$\frac{dy}{dx} = const \Leftrightarrow \frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1}$$

$$\Delta y = y_i - y_{i-1}$$

$$y_i = y_{i-1} + \Delta y = y_{i-1} + \frac{y_2 - y_1}{x_2 - x_1}\Delta x = y_{i-1} + \frac{Dy}{Dx}$$

Цифровой дифференциальный анализатор (ЦДА)

Несимметричный алгоритм ЦДА (I октант, Dy<Dx)

Вычислить приращения координат:

```
Dx= x2 - x1;
Dy= y2 - y1;
```

Занести начальную точку отрезка

```
PutPixel (x1, y1);
```

Сгенерировать отрезок

```
while (x1 < x2) {
x1:= x1 + 1.0;
y1:= y1 + Dy/Dx;
PutPixel (x1, y1);
}
```

Цифровой дифференциальный анализатор (ЦДА)

Несимметричный алгоритм ЦДА (Dy<Dx)

Вопрос вычисления целочисленной координаты *y* на каждой итерации

- округление;
- отбрасывание дробной части;

Цифровой дифференциальный анализатор (ЦДА)

Несимметричный алгоритм ЦДА (II октант, Dx<Dy)

Вычислить приращения координат:

```
Dx= x2 - x1;
Dy= y2 - y1;
```

Занести начальную точку отрезка

```
PutPixel (x1, y1);
```

Сгенерировать отрезок

```
while (y1 < y2) {
y1:= y1 + 1.0;
x1:= x1 + Dx/Dy;
PutPixel (x1, y1);
}
```

Цифровой дифференциальный анализатор (ЦДА)

Симметричный алгоритм ЦДА (I четверть, x1<x2,y1<y2)

Вычислить максимум из приращений координат:

```
Dx= x2 - x1;    Dy= y2 - y1;
L=max {Dx, Dy};
```

Занести начальную точку отрезка

```
PutPixel (x1, y1);
```

Сгенерировать отрезок

```
i=0;
while (i<=L) {
y1:= y1 + Dy/L;
x1:= x1 + Dx/L;
PutPixel (x1, y1); ++i;
}
```



Алгоритм Кастла-Питвея

- Строит 8-связную линию.
- Гораздо менее эффективен с вычислительной точки зрения, чем алгоритм Брезенхема.
- Обладает красивой математической структурой и представляет интерес с чисто алгоритмической точки зрения.
- Основан на идее, схожей с известным алгоритмом Евклида нахождения НОД двух натуральных чисел.

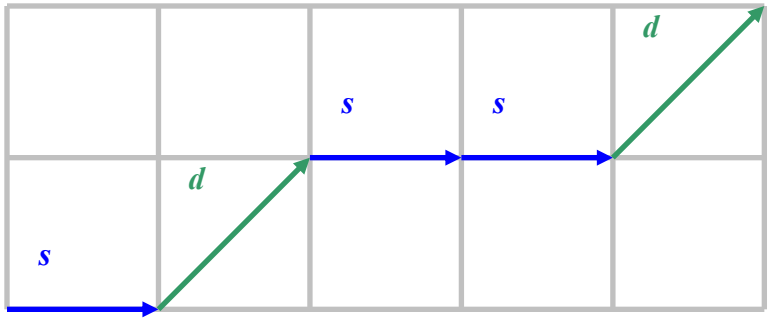
Алгоритм Кастла-Питвея

В каноническом случае процесс рисования 8-связной линии можно закодировать последовательностью вида: sdssd…

Где:

s - горизонтальное смещение;

d - диагональное смещение.





Алгоритм Кастла-Питвея

Рисуем отрезок (0,0)-(a,b)

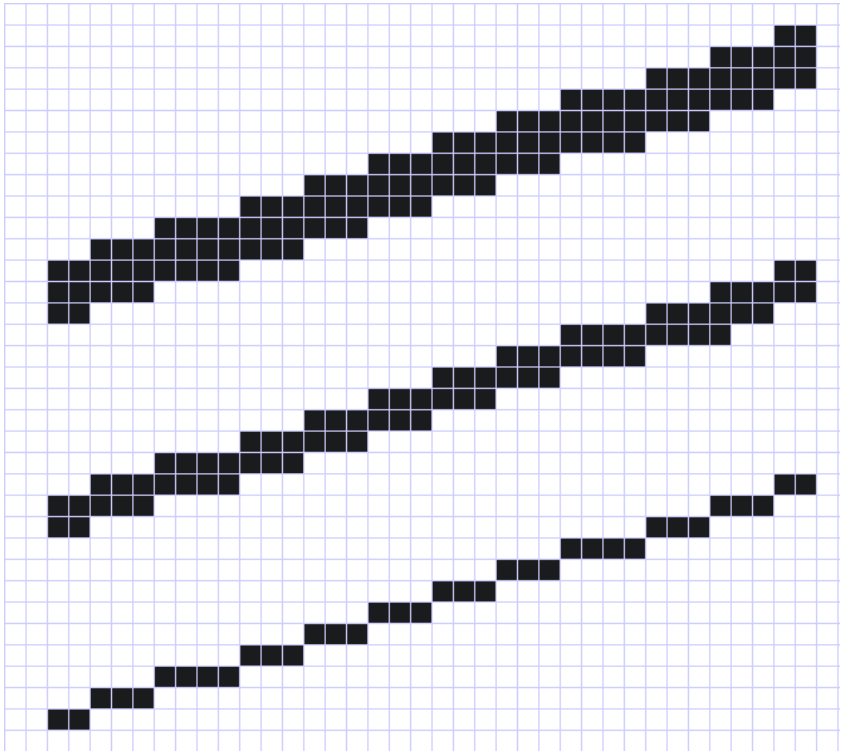
```
y = b;
x = a - b;
m1 = "s";   m2 = "d";
while( x <>y ) {
        if( x > y ) {
                x = x - y;
                m2 = m1 (+) ~ m2;
        } else {
                y = y - x;
                m1 = m2 (+) ~ m1;
        }
}
```

После завершения работы алгоритма ***x*-кратная** последовательность **m2 (+) ~ m1** задает нужный отрезок.

Изображение отрезков толщиной в более чем один пиксель

Алгоритм: использование фигуры (например, квадрата) или линии в качестве элемента вывода.

Пример: линия толщины 3: Закрашивается очередной пиксель (x_i,y_i), а также пиксели (x_i,y_i+1) и (x_i,y_i+2).



Изображение отрезков с нецелочисленными координатами концов

1 вариант:

Округление координат до целочисленных.

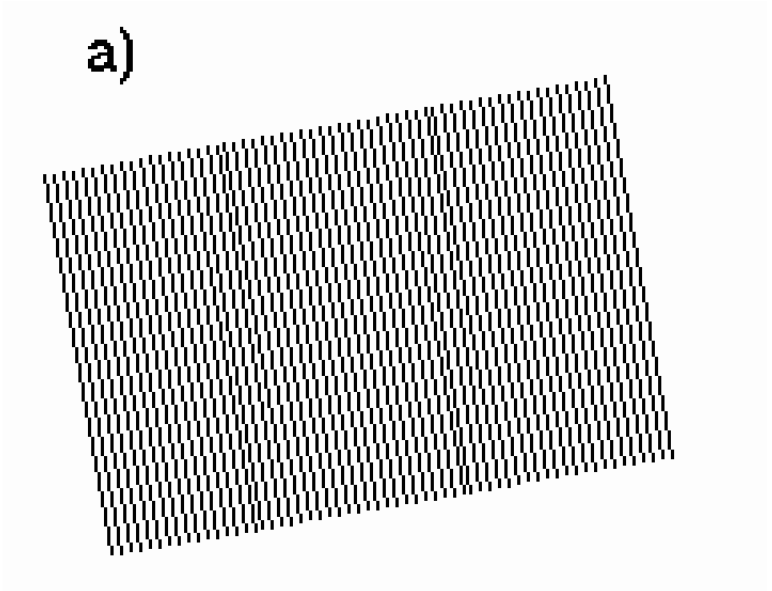
Недостатки: могут возникнуть искажения (особенно в случае отрезков небольшой длины).

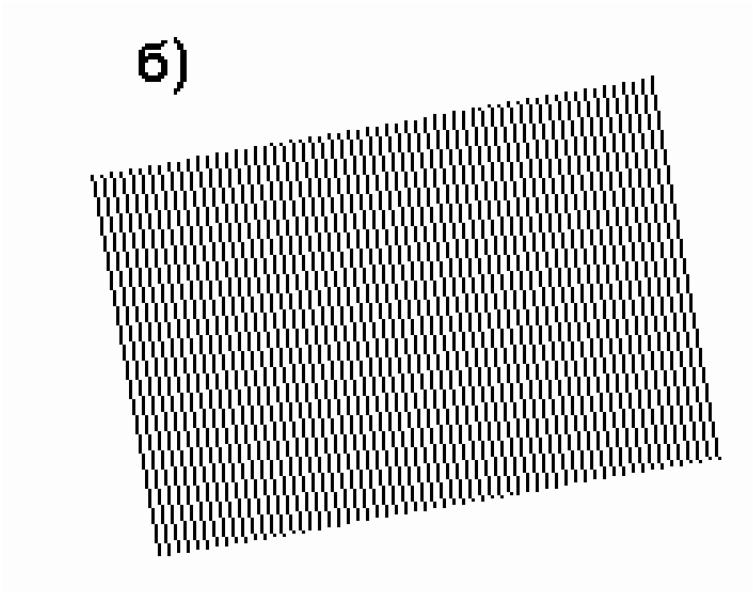
2 вариант

Алгоритм растеризации с субпиксельной точностью

Величина ошибки вычисляется с поправкой на дробную часть координат.

Изображение отрезков с нецелочисленными координатами концов







Рисование окружностей и эллипсов

Во многих областях базовыми графическими примитивами, помимо отрезков прямых, являются и конические сечения, т.е. окружности, эллипсы, параболы и гиперболы. Наиболее употребительным примитивом является окружность. Один из наиболее простых и эффективных алгоритмов генерации окружности также разработан Брезенхемом.

Рисование окружностей и эллипсов

Уравнение окружности радиуса R с центром в точке (x_0,y_0)

$$(x-x_0)^2+(y-y_0)^2=R^2$$

Пошаговое вычисление координат

$$y_i=y_0+\sqrt{R^2-(x_i-x_0)^2}$$

$$x_i=x_0-R,\ldots,x_0+R$$

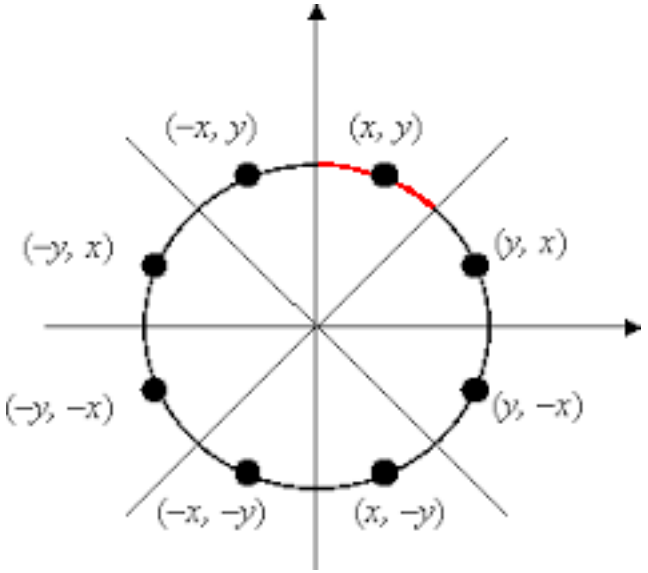
Алгоритм Брезенхема

для построения окружности

Рассмотрим работу алгоритма на примере построения окружности с центром в начале координат.

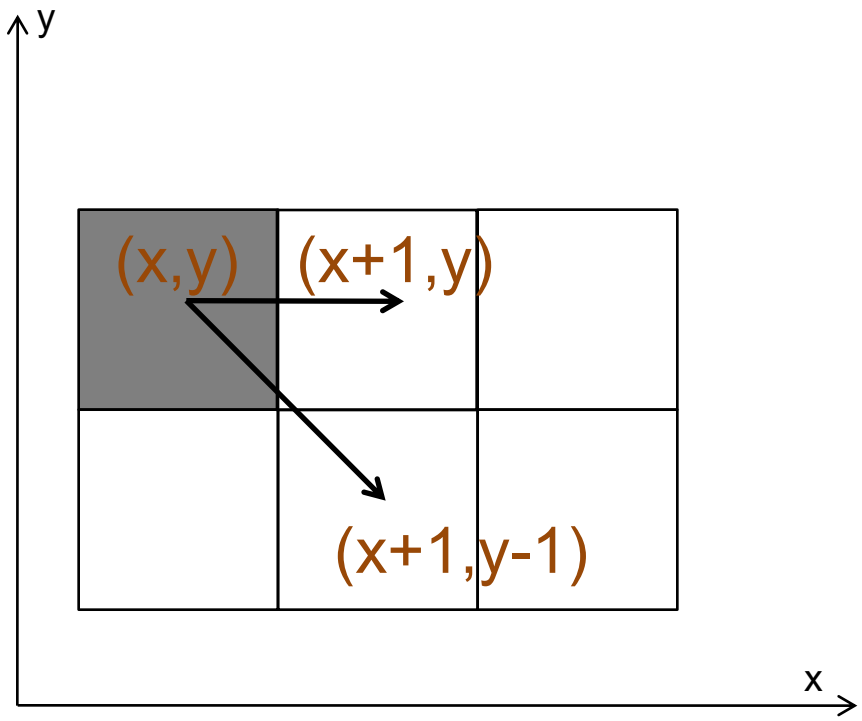
Достаточно построить одну дугу окружности, расположенную во втором октанте. Координаты точек окружности для остальных октантов получаются симметрично.

На каждом шаге алгоритма в качестве ошибки выбирается расстояние от окружности до ближайших координат сетки.



Алгоритм Брезенхема

для построения окружности



Начинаем из точки: $(0; R)$

Завершаем в точке: $\left(\frac{R\sqrt{2}}{2}; \frac{R\sqrt{2}}{2}\right)$

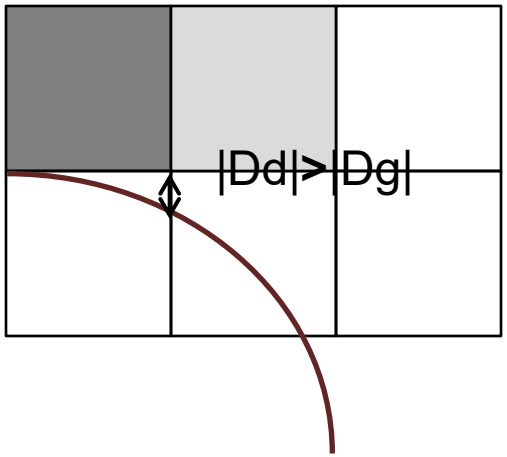
На каждой итерации из текущих координат (x,y) возможны два смещения – в точку (x+1,y) либо в точку (x+1,y-1).

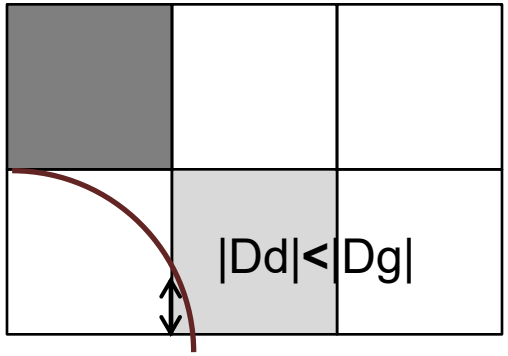
Алгоритм Брезенхема

для построения окружности

$$Dg = (x+1)^2 + y^2 - R^2$$

$$Dd = (x+1)^2 + (y-1)^2 - R^2$$





Целочисленный алгоритм Брезенхема для дуги окружности

```
X= 0;
Y= R;
E= 3-2*R;
```

Первая точка вектора

```
PutPixel(X, Y);
```

```
while (X<Y) {
        if (E ⩾0) {
                E= E + 4*(X-Y)+10;
                X= X+ 1;
                Y= Y - 1;
        } else {
                E= E + 4*X+6;
                X= X + 1;
                }
```

Очередная точка вектора

```
        PutPixel(X, Y);
        }
```

Целочисленный алгоритм Брезенхема для целой окружности

```
PutPixel(X, Y);    // отрисовали точку дуги II октанта
PutPixel(X, -Y);   // симметричная точка VII октанта
PutPixel(-X, Y);   // симметричная точка III октанта
PutPixel(-X, -Y);  // симметричная точка V октанта

PutPixel(Y, X);    // симметричная точка I октанта
PutPixel(Y, -X);   // симметричная точка VIII октанта
PutPixel(-Y, X);   // симметричная точка IV октанта
PutPixel(-Y, -X);  // симметричная точка VI октанта
```

Целочисленный алгоритм Брезенхема для окружности с центром в произвольной точке (X0,Y0)

```
PutPixel(X + X0, Y + Y0);
PutPixel(X + X0, - Y + Y0);
PutPixel(-X + X0, Y + Y0);
PutPixel(-X + X0, - Y + Y0);

PutPixel(Y + X0, X + Y0);
PutPixel(Y + X0, -X + Y0);
PutPixel(-Y + X0, X + Y0);
PutPixel(-Y + X0, -X + Y0);
```